GraphGuard_AML

GraphGuard AML: Explainable Detection of Suspicious Transaction Networks

Project Research & Implementation Dossier

Working project name: GraphGuard AML
Project type: Finance + Graph Machine Learning + Explainable AI + Financial Crime Detection
Target team: 5 people
Target build window: 12 weeks for a complete MVP, with the fourth month reserved for hardening, experiments, documentation and presentation
Primary dataset: IBM AMLSim synthetic banking transaction data
Secondary dataset: Elliptic Bitcoin transaction graph
Primary ML direction: XGBoost baseline + GraphSAGE/GAT-class GNN
Primary XAI direction: GNNExplainer for graph structure + SHAP for engineered/tabular graph features
Primary product output: Analyst-style AML case investigation dashboard

---

1. Executive Summary

1.1 What are we building?

GraphGuard AML is an explainable anti-money-laundering investigation system that turns a stream of financial transactions into a transaction graph, identifies accounts or transaction groups that exhibit suspicious behavior, and presents an explanation of the suspicious pattern to a human investigator.

The system is intentionally not framed as an autonomous system that declares that a customer is committing a crime. Its output is an investigation risk signal and an evidence-oriented explanation that helps an analyst decide what deserves further review.

The core pipeline is:

```
Transaction Data
      |
      v
Data Validation & Cleaning
      |
      v
Entity Resolution / Graph Construction
      |
      v
Graph + Temporal Feature Engineering
      |
      +------------------------+
```

```
      |                        |
      v                        v
Tabular Baseline          Graph Model
XGBoost / RF              GraphSAGE / GAT
      |                        |
      |                        v
      |                  Node / Edge Risk
      |                        |
      +-----------+------------+
                  |
                  v
             Explanation
        +---------+---------+
        |                   |
        v                   v
      SHAP             GNNExplainer
        |                   |
        +---------+---------+
                  |
                  v
          Case Construction
                  |
                  v
        Investigator Dashboard
```

1.2 Why this is a real problem

Conventional AML systems are heavily rule-driven. A rule fires, an alert is created, and a human investigates it. A major operational problem is that AML alerting can produce large numbers of false positives, making investigation expensive and time-consuming.

The project literature directly supports this framing. The 2021 paper *Anti-Money Laundering Alert Optimization Using Machine Learning with Graphs* describes existing rule-based AML workflows, high false-positive rates, expensive human review, and the use of graph-based features and temporal windows to improve alert triage. (arXiv)

The 2022 *LaundroGraph* paper frames AML investigation as navigating a large network of financial interactions and proposes self-supervised graph representation learning to surface useful insights for analysts. (arXiv)

The finance literature supplied for this project also identifies AML as comparatively understudied within XAI-in-finance research, while highlighting the need for explainability, traceability and useful explanations in high-impact financial decisions. See the uploaded finance review and project notes.

1.3 The central project question

> Can graph-based machine learning identify suspicious transaction networks and produce compact, human-understandable explanations that help an AML investigator decide which alerts deserve deeper investigation?

This is deliberately a systems-and-evaluation question, not a demand to invent a new GNN architecture.

1.4 What makes the project more than "GNN + SHAP"

The novelty should come from the end-to-end investigation workflow:

1. represent financial activity as a graph;
2. model structural and temporal behavior;
3. detect suspicious entities or transactions;
4. explain the prediction using graph structure and feature contributions;
5. turn those explanations into a coherent investigation case;
6. evaluate not just classification quality, but explanation quality and analyst usefulness.

The final demo should let an examiner select a suspicious account and see the surrounding transaction network, risk score, important features, suspicious paths/cycles, and a concise evidence summary.

---

2. Scope Boundary

2.1 Core scope

The minimum complete system should include:

- synthetic banking transaction generation using AMLSim;
- transaction graph construction;
- account-level and transaction-level feature engineering;
- a non-graph baseline such as XGBoost;
- one primary GNN, preferably GraphSAGE;
- graph-aware explanation using GNNExplainer;
- SHAP for tabular/engineered graph features;
- class-imbalance-aware evaluation;
- an investigator dashboard;
- reproducible experiments.

2.2 Explicitly optional scope

The following are stretch items, not requirements:

- GAT;
- heterogeneous graphs;
- temporal GNNs;
- self-supervised pretraining;
- multiple AML typologies simultaneously;
- advanced counterfactual explanations;

- graph-level explanation benchmarking;
- analyst user study;
- live-stream ingestion;
- cryptocurrency data.

2.3 Things the team should NOT make mandatory

Do not make these part of the first implementation:

- custom cryptography;
- secure multi-party computation;
- blockchain infrastructure;
- a novel GNN architecture invented from scratch;
- a large language model as the core detector;
- production banking integration;
- real customer data;
- Docker-only deployment.

Those are all respectable technologies. They are also efficient ways to turn a 12-week project into a collection of unfinished README files.

---

3. Research Background

3.1 XAI in finance

The uploaded finance literature review reports that explainability in finance is motivated by transparency, traceability, trust, fairness and regulatory needs. It reviews 2,022 initial articles and narrows them to 60 relevant studies, finding that risk management, stock-market applications and portfolio optimization were better represented than AML and electronic financial transaction classification.

The same review identifies nine XAI goals:

- trustworthiness;
- causality;
- transferability;
- informativeness;
- confidence;
- fairness;
- accessibility;
- interactivity;
- privacy awareness.

It notes that informativeness and confidence receive much more attention than privacy awareness, causality and accessibility. These observations are useful for the project

because GraphGuard can focus on informativeness + interactivity + auditability without pretending to solve all responsible-AI problems.

Source: uploaded `xAI in Finance.md` and `Research-able_Projects?[Finance].md`.

3.2 XAI in cybersecurity and financial crime

The uploaded cybersecurity survey explicitly lists fraud detection as an XAI application and names:

- SHAP;
- LIME;
- GNNExplainer;
- counterfactual explanations;
- graph-oriented methods.

It also describes a common XAI workflow: define the security task, collect data, preprocess, extract features, train a model, generate explanations, and present the decision plus explanation to a human user.

Source: uploaded `xAI in Cyber Security.md`.

3.3 Why graphs?

A financial transaction is relational by nature.

A tabular row may tell us:

```
sender = A
receiver = B
amount = 90000
hour = 17
```

But laundering investigations often care about relationships such as:

```
A → B → C → D
|         |
+-> E <---+
```

The graph representation makes it possible to model:

- number of counterparties;
- repeated fund transfers;
- fan-in and fan-out structures;
- cycles;
- communities;
- short and long transaction paths;
- concentration of funds;
- relationships between account types/entities;

- temporal changes in behavior.

That is the central reason graph ML is appropriate for this project.

---

4. Research Papers and Technical References Visited

This section distinguishes between papers that directly motivate the AML system and foundational papers needed to implement it.

4.1 Direct AML / financial-graph papers

Paper 1 —Scalable Graph Learning for Anti-Money Laundering: A First Look

Authors: Mark Weber, Jie Chen, Toyotaro Suzumura, Aldo Pareja, Tengfei Ma, Hiroki Kanezashi, Tim Kaler, Charles E. Leiserson, Tao B. Schardl
Year: 2018
Venue/source: arXiv / IBM research
Link: https://arxiv.org/abs/1812.00076
Access: Free full text

Why it matters:

This is one of the foundational papers connecting large-scale graph deep learning with AML. It explains why AML data is massive, dense and dynamic, and reports experiments on a large synthetic graph generated with AMLSim.

What to borrow:

- graph representation of AML activity;
- scale awareness;
- AMLSim as a synthetic-data source;
- dynamic graph framing;
- need for computationally efficient graph learning.

What not to copy directly:

The paper's scale is much larger than what is required for a student MVP. We can use AMLSim on a smaller configuration first and demonstrate scaling as an experiment.

Paper 2 —Anti-Money Laundering Alert Optimization Using Machine Learning with Graphs

Authors: Ahmad Naser Eddin, Jacopo Bono, David Aparício, David Polido, João Tiago Ascensão, Pedro Bizarro, Pedro Ribeiro
Year: 2021/2022 version

Link: https://arxiv.org/abs/2112.07508
Access: Free full text

Why it matters:

This is probably the most important paper for the operational side of GraphGuard. Rather than replacing an AML rules engine completely, it trains a machine-learning triage model that uses entity-centric and graph-based features, including temporal windows, to prioritize alerts.

The paper reports an experiment on real banking data in which the proposed triage approach reduced false positives while retaining strong true-positive detection. Treat these numerical results as results of their dataset and experimental setup, not as a target for our project.

What to borrow:

- AML as an alert-triage problem;
- graph-based relational features;
- temporal windows;
- operational metrics such as false-positive reduction;
- complementing existing rule systems instead of pretending ML replaces them overnight.

Direct design influence on GraphGuard:

The GraphGuard dashboard should eventually show both the model's risk score and the evidence that made the alert worth investigating.

Paper 3 —LaundroGraph: Self-Supervised Graph Representation Learning for Anti-Money Laundering

Authors: Mário Cardoso, Pedro Saleiro, Pedro Bizarro
Year: 2022
Venue: ACM ICAIF workshop
Link: https://arxiv.org/abs/2210.14360
Access: Free arXiv version

Why it matters:

AML labels are scarce. LaundroGraph addresses this by learning useful graph representations without requiring a fully supervised training label for every example.

It models a customer-transaction bipartite graph and uses self-supervised link prediction. The goal is to generate representations and analyst-facing insights.

What to borrow:

- awareness that labeled laundering examples are scarce;
- bipartite customer/transaction representation;

- self-supervised learning as a stretch direction;
- analyst assistance rather than pure classification.

What not to make core:

Self-supervised pretraining adds complexity. Use it only after the supervised GraphSAGE MVP is stable.

Paper 4 —DELATOR: Money Laundering Detection via Multi-Task Learning on Large Transaction Graphs

Authors: Henrique S. Assumpção et al.
Year: 2022
Link: https://arxiv.org/abs/2205.10293
Access: Free full text

Why it matters:

This work focuses on large temporal transaction graphs, class imbalance and multi-task learning. It is useful for understanding advanced directions beyond a simple node-classification GNN.

What to borrow:

- temporal transaction graphs;
- serious treatment of class imbalance;
- why plain supervised learning can struggle with AML labels;
- the idea of learning multiple useful signals rather than one binary label.

Project position:

Reference for the discussion/advanced work. Not required for the MVP.

Paper 5 —Graph-Based LSTM for Anti-money Laundering: Experimenting Temporal Graph Convolutional Network with Bitcoin Data

Authors: Ismail Alarab, Simant Prakoonwit
Publication: Neural Processing Letters, 2023
Link: https://link.springer.com/article/10.1007/s11063-022-10904-8
Access: Open access

Why it matters:

This paper explores temporal graph modeling for AML using Bitcoin transaction data. It is useful when thinking about how money flows evolve over time rather than treating the graph as static.

What to borrow:

- temporal windows as a first step;
- graph + sequence modeling as a future extension;
- Bitcoin/Elliptic as a secondary benchmark.

What not to make core:

A temporal GNN/LSTM architecture should be a stretch goal because it increases both implementation and evaluation complexity.

Paper 6 —Catch Me If You Can: Semi-supervised Graph Learning for Spotting Money Laundering

Authors: Md. Rezaul Karim, Felix Hermsen, Sisay Adugna Chala, Paola de Perthuis, Avikarsha Mandal
Year: 2023
Link: https://arxiv.org/abs/2302.11880
Access: Free full text

Why it matters:

This paper explores semi-supervised graph learning for AML and emphasizes the fact that laundering can involve chains of accounts and high-volume flows rather than simple local anomalies.

What to borrow:

- account-chain perspective;
- semi-supervised learning as a future extension;
- need to capture flow structure.

Paper 7 —Finding Money Launderers Using Heterogeneous Graph Neural Networks

Authors: Fredrik Johannessen, Martin Jullum
Year: 2023 preprint; later open-access journal publication 2025
arXiv: https://arxiv.org/abs/2307.13499
Journal: https://www.sciencedirect.com/science/article/pii/S2405918825000273
Access: Free arXiv version; journal version open access

Why it matters:

Real financial systems are heterogeneous. Accounts, people, organizations, roles and transaction types do not all represent the same kind of node or edge.

This paper extends message passing to heterogeneous AML graphs using real bank transaction and business-role data from DNB Norway.

What to borrow:

- heterogeneous graph thinking;
- business-role relations;
- PyTorch Geometric implementation reference;
- motivation for moving beyond homogeneous account graphs.

Project position:

Use homogeneous account/transaction graphs for the MVP. Make heterogeneous modeling a stretch goal.

Paper 8 —LineMVGNN: Anti-Money Laundering with Line-Graph-Assisted Multi-View Graph Neural Networks

Authors: Chung-Hoo Poon, James Kwok, Calvin Chow, Jang-Hyeon Choi
Year: 2025/2026 versions
arXiv: https://arxiv.org/abs/2603.23584
Open-access journal page: https://www.mdpi.com/2673-2688/6/4/69
Access: Open access

Why it matters:

This is a useful recent reference because it explicitly discusses directed transaction graphs, multi-dimensional edge features, scalability and interpretability concerns.

What to borrow:

- payment vs receipt views;
- explicit treatment of edge information;
- line-graph perspective as an advanced direction;
- attention to scalability and explainability.

Do not copy the architecture for the MVP. It is a reference for future experimentation.

Paper 9 —Synthetic Pattern Generation and Detection of Financial Activities using Graph Autoencoders

Year: 2026
Journal: Procedia Computer Science
Link: https://www.sciencedirect.com/science/article/pii/S1877050926011798
Access: Open access

Why it matters:

The paper is very relevant to AMLSim-style synthetic pattern generation: illicit financial behavior is represented through recurring graph structures, and graph autoencoders are used to learn such patterns.

What to borrow:

- graph-pattern interpretation;
- synthetic-data experimentation;
- unsupervised/semi-supervised anomaly detection as future work.

Paper 10 —HeteroGCL: A Heterogeneous Graph Contrastive Learning Framework for Scalable and Sustainable Cryptocurrency AML

Year: 2026
Journal: Applied Sciences
Link: https://www.mdpi.com/2076-3417/16/6/2860
Access: Open access

Why it matters:

This is a very recent reference on heterogeneous graph contrastive learning for cryptocurrency AML. It focuses on class imbalance, complex transaction relationships and limited labels.

Project position:

Use as a recent-state-of-the-art reference in the literature review. Do not make contrastive learning part of the first MVP.

Paper 11 —Graph Contrastive Pre-training for Anti-money Laundering

Authors: Hanbin Lu, Haosen Wang
Year: 2024
Journal: International Journal of Computational Intelligence Systems
Link: https://link.springer.com/article/10.1007/s44196-024-00720-4
Access: Open access

Why it matters:

This paper uses graph contrastive pretraining to reduce dependence on scarce AML labels. It is a useful bridge between fully supervised GraphSAGE and the advanced self-supervised directions seen in AML research.

Project position:

Stretch / future-work reference.

Paper 12 —Digital currency money laundering identification model based on graph structure and transformer self-supervised learning

Year: 2026
Journal: Discover Artificial Intelligence
Link: https://link.springer.com/article/10.1007/s44163-026-02264-2
Access: Open access

Why it matters:

This is a very recent example of moving from graph structure toward graph-derived sequences and Transformer/self-supervised modeling for AML under limited-label conditions.

Project position:

Use only in the state-of-the-art discussion. A Transformer is far beyond the required scope of the student MVP.

---

5. Foundational Graph / XAI Papers

5.1 GraphSAGE

Paper: *Inductive Representation Learning on Large Graphs*
Authors: William L. Hamilton, Rex Ying, Jure Leskovec
Year: 2017
Link: https://arxiv.org/abs/1706.02216
Implementation: PyTorch Geometric `SAGEConv`

Core idea

Instead of learning a permanent embedding for each node, GraphSAGE learns a function that aggregates information from a node and its neighborhood. This makes it attractive for settings where new nodes can appear after training.

A simplified formulation is:

```
h_i^(k) = COMBINE(
              h_i^(k-1),
              AGGREGATE({h_j^(k-1) : j in N(i)})
          )
```

A simple mean-aggregation version is supported directly by PyTorch Geometric. PyG SAGEConv documentation

Why GraphSAGE fits GraphGuard

It gives us:

- a well-established GNN architecture;
- manageable implementation complexity;
- an inductive formulation;
- direct support in PyTorch Geometric;
- a clean baseline against XGBoost.

5.2 GNNExplainer

Paper: *GNNExplainer: Generating Explanations for Graph Neural Networks*
Authors: Rex Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, Jure Leskovec
Year: 2019
Link: https://arxiv.org/abs/1903.03894
Official code: https://github.com/RexYing/gnn-model-explainer

Core idea

GNNExplainer searches for a compact subgraph and a subset of node features that are important for a GNN prediction.

For GraphGuard, this means the system can answer:

> "Which neighboring accounts and transactions were most relevant to this suspicious-risk prediction?"

That is much closer to what an AML analyst needs than a global feature bar chart.

Important distinction

SHAP answers approximately:

> Which numerical features pushed this prediction?

GNNExplainer answers approximately:

> Which part of the graph helped produce this prediction?

Use both.

5.3 SHAP

Paper: *A Unified Approach to Interpreting Model Predictions*
Authors: Scott M. Lundberg, Su-In Lee
Year: 2017
Link: https://arxiv.org/abs/1705.07874

SHAP is best used in GraphGuard on engineered tabular features such as:

- transaction count;
- total inbound amount;
- total outbound amount;
- number of counterparties;
- average transaction amount;
- account balance change;
- cycle count;
- graph centrality features;
- recent transaction intensity.

The explanation can then be translated into a human-facing sentence template.

5.4 LIME

Paper: *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*
Authors: Marco Tulio Ribeiro, Sameer Singh, Carlos Guestrin
Year: 2016
Link: https://arxiv.org/abs/1602.04938

LIME is useful as a comparison method for local explanations, but it is not necessary for the MVP.

A reasonable research experiment is:

```
SHAP explanation
       vs
LIME explanation
       vs
GNNExplainer subgraph
```

Do not turn this into a giant "all XAI methods ever invented" benchmark.

---

6. Datasets

6.1 Primary dataset: IBM AMLSim

Repository: https://github.com/IBM/AMLSim

AMLSim is a multi-agent simulator designed to generate synthetic banking transactions together with known money-laundering patterns. It is specifically intended for researchers working on ML models and graph algorithms.

Why AMLSim should be the primary dataset

- known synthetic AML patterns;
- repeatable experiments;
- controllable scale;
- no exposure of private banking data;
- the transaction graph can be generated directly;
- labels are linked to known alert/suspicious patterns;
- useful for testing explanation correctness.

The IBM repository documents sample configurations including 1K and 10K-style simulation setups and outputs such as transactions, alert accounts, alert transactions and SAR-related files.

Important AMLSim data concepts

AMLSim's `alertPatterns.csv` defines typology-level parameters such as:

- typology type;
- scheduling behavior;
- minimum/maximum accounts involved;
- minimum/maximum transaction amount;
- minimum/maximum period;
- bank identifier;
- whether the pattern maps to a SAR or false alert.

The project therefore gives us a major advantage: we know the synthetic ground-truth pattern used to generate an alert.

Relevant AML patterns

AMLSim documents examples including patterns such as:

- fan-in;
- fan-out;
- cycle;
- scheduled transaction chains;
- transaction behaviors involving different intervals.

These patterns are extremely useful for evaluating whether GNNExplainer actually identifies meaningful graph structure.

AMLSim repository notes

The repository currently recommends using the master branch for practical use. Its documented tooling uses Python and Java/MASON. The README contains scripts for graph generation, simulation, log conversion, visualization and alert validation.

Repository: https://github.com/IBM/AMLSim

Parameter documentation: https://github.com/IBM/AMLSim/blob/master/paramFiles/README.md

Sample 10K configuration:
https://github.com/IBM/AMLSim/blob/master/paramFiles/10K/conf.json

---

6.2 Secondary dataset: Elliptic Bitcoin Transaction Dataset

Dataset page: https://www.kaggle.com/ellipticco/elliptic-data-set

Dataset characteristics

The published dataset describes an anonymized Bitcoin transaction graph with:

- 203,769 nodes;
- 234,355 edges;
- 166 node features;
- licit, illicit and unknown labels;
- 49 time steps;
- roughly two-week intervals between time steps.

Why use it

The Elliptic dataset gives the project an independently sourced graph dataset rather than relying exclusively on data generated by our own simulator.

Why NOT make it the primary dataset

AMLSim offers known synthetic AML typologies that are better suited to validating explanations. Elliptic is excellent for a secondary generalization experiment, but its feature semantics and label distribution are not as convenient for explaining synthetic laundering patterns.

Suggested use

Use Elliptic during Weeks 9–11 only if the core AMLSim pipeline is already working.

---

7. GitHub Repositories and Software Resources

7.1 IBM AMLSim

https://github.com/IBM/AMLSim

Role: synthetic AML generator and transaction simulator.

Use it for:

- data generation;
- known AML pattern creation;
- graph construction reference;

- data validation;
- transaction simulation.

7.2 Heterogeneous MPNN implementation

https://github.com/fredjo89/heterogeneous-mpnn

This repository contains an implementation of a heterogeneous message-passing neural network used in the heterogeneous AML paper.

Use it for:

- understanding a heterogeneous AML implementation;
- PyTorch Geometric usage;
- future extension.

Do not use it as the main project architecture.

7.3 Official GNNExplainer repository

https://github.com/RexYing/gnn-model-explainer

Use it for:

- understanding the original GNNExplainer implementation;
- reproducing small examples;
- understanding explanation masks.

For current project implementation, PyTorch Geometric's current explanation APIs may be easier to integrate than the original research repository.

7.4 PyTorch Geometric

https://pytorch-geometric.readthedocs.io/

Use it for:

- `Data` / `HeteroData` graph objects;
- `SAGEConv`;
- GCN/GAT layers;
- graph sampling;
- explanation tooling;
- batching.

7.5 Graph / fraud research bibliography

https://github.com/safe-graph/graph-fraud-detection-papers

This is a useful literature index containing graph-based fraud/anomaly work and references to papers such as LaundroGraph.

Use it for: literature discovery, not as the authority for the project's claims.

7.6 GNN-for-AML paper list

https://github.com/fuqishuang228/GNN-for-Anti-money-launedring

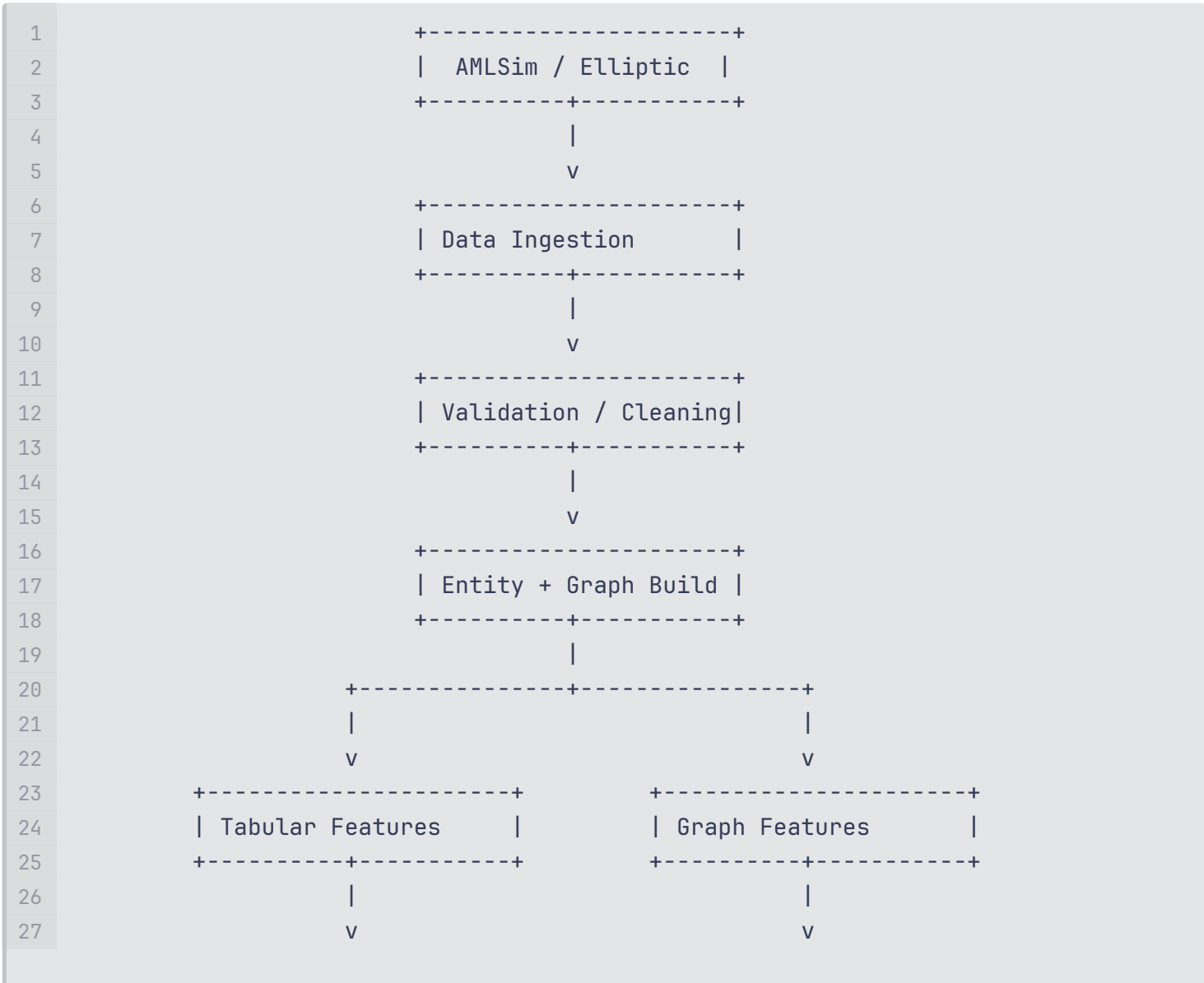
This repository curates papers and code on graph-based anti-money-laundering research.

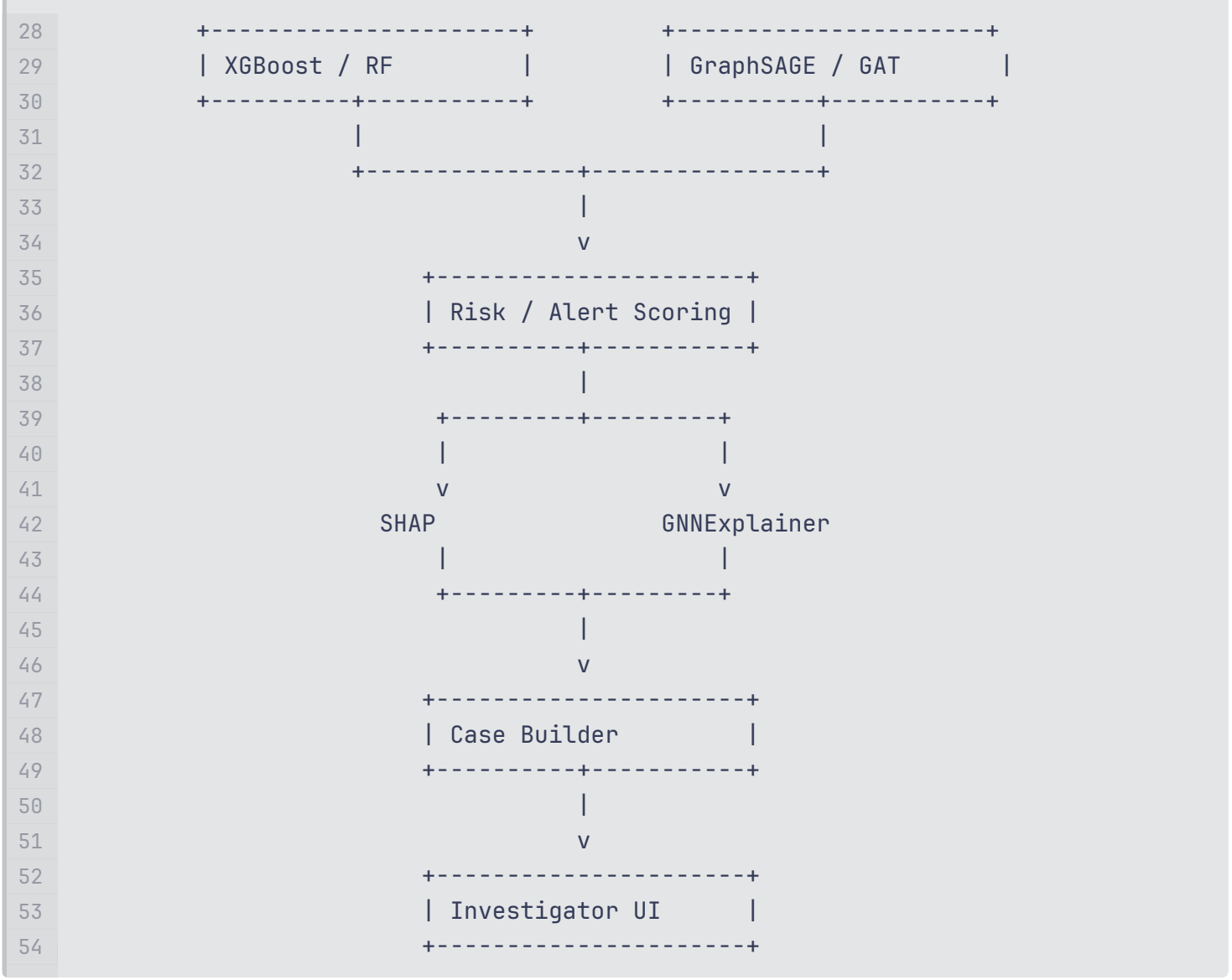
Use it for: discovering adjacent work and locating alternative implementations.

---

8. Proposed System Architecture

8.1 High-level architecture





---

9. Module-by-Module Design

Module 01 —Data Ingestion

Purpose

Read raw AMLSim transaction outputs or a secondary dataset into a normalized internal format.

Input

At minimum:

```
transaction_id
sender_account
receiver_account
timestamp
amount
transaction_type
bank_id
```

Depending on AMLSim configuration, additional fields may exist.

Output

A normalized transaction table:

```
transactions.parquet
```

Responsibilities

- load CSV/Parquet;
- normalize column names;
- parse timestamps;
- convert amounts to numeric;
- validate identifiers;
- record dataset statistics.

Tests

- no duplicate transaction IDs;
- no impossible negative amounts unless the source explicitly defines them;
- sender/receiver IDs are valid;
- timestamps are parseable;
- deterministic preprocessing given a fixed seed.

Module 02 —Data Validation and Cleaning

Purpose

Prevent garbage data from becoming apparently sophisticated garbage.

Operations

- missing-value handling;
- invalid timestamp handling;
- duplicate detection;
- disconnected-node handling;
- amount outlier diagnostics;
- label consistency checks;
- timestamp ordering checks.

Important design decision

Do not remove extreme transactions blindly. In AML, unusual amounts may be the very signal we need.

Instead:

1. report outliers;
2. decide whether they are invalid records or legitimate rare behaviors;
3. preserve legitimate extremes.

Output

```
clean_transactions.parquet
validation_report.json
```

Module 03 —Graph Construction

Purpose

Transform financial events into a graph structure.

Basic graph

For the MVP:

- node = account
- directed edge = transaction
- edge amount = transaction amount
- edge timestamp = transaction time
- edge type = transaction type

Example:

```
Account A ----₹10k----> Account B
Account B ----₹9k-----> Account C
Account C ----₹8k-----> Account A
```

Alternative bipartite graph

A second representation can be tested:

```
Account A ---- Transaction 101 ---- Account B
```

Here:

- account = one node type;
- transaction = another node type.

This is closer to LaundroGraph's customer-transaction bipartite formulation.

MVP choice

Use the account graph first.

Use bipartite modeling only as an advanced experiment.

Output

For PyTorch Geometric:

```
Data(
    x=node_features,
    edge_index=edge_index,
    edge_attr=edge_features,
    y=node_labels
)
```

Module 04 —Temporal Window Builder

Purpose

Avoid treating a six-month-old transaction and a transaction from five minutes ago as if they are equally informative.

Example windows

```
W1 = 1 day
W2 = 7 days
W3 = 30 days
```

Within each window calculate:

- number of outgoing transactions;
- number of incoming transactions;
- total incoming amount;
- total outgoing amount;
- unique counterparties;
- average amount;
- amount variance;
- transaction burstiness;
- time since previous transaction.

Why this matters

The AML alert optimization paper explicitly uses temporal windows to construct dynamic graph information efficiently. (arXiv)

Module 05 —Account-Level Feature Engineering

Purpose

Create interpretable features for XGBoost and to provide an explanation layer alongside the GNN.

Core features

Activity

- transaction_count;
- active_days;
- active_hours;
- recent_transaction_count.

Money flow

- total_in;
- total_out;
- in_out_ratio;
- mean_transaction;
- median_transaction;
- max_transaction;
- transaction_std.

Counterparty behavior

- unique_in_counterparties;
- unique_out_counterparties;
- counterparty_entropy;
- repeat_counterparty_ratio.

Graph structure

- in_degree;
- out_degree;
- weighted_degree;
- PageRank;
- clustering coefficient where appropriate;
- connected-component size;
- cycle_count;
- k-hop neighborhood size.

Temporal behavior

- transaction_burst_score;
- amount_change_rate;
- new_counterparty_rate;

- short-time fan-in;
- short-time fan-out.

Important principle

Every engineered feature should have a plain-English interpretation.

For example:

```
out_degree = number of distinct destination accounts
```

is explainable.

A mysterious 512-dimensional embedding is not.

Module 06 —AML Typology / Ground-Truth Mapper

Purpose

Turn AMLSim's known patterns into labels useful for training and evaluation.

Example

```
cycle
fan_in
fan_out
smurfing-like pattern
scheduled chain
```

The exact typology names should be taken from the specific AMLSim configuration being used rather than invented by the team.

Output

```
account_labels.csv
transaction_labels.csv
pattern_memberships.csv
```

Why this module matters

It allows us to evaluate explanations against known generated patterns.

For example:

> AMLSim generated a cycle involving accounts A, B, C and D.

If GraphSAGE flags B and GNNExplainer highlights A–B–C–D–A, that is qualitatively useful evidence.

Module 07 —Baseline Rule Engine

Purpose

Create a simple non-ML benchmark.

Example rules

```
if out_degree > threshold → alert
if burst_score > threshold → alert
if cycle_count > threshold → alert
if amount_ratio unusual → alert
```

Why build it?

Without a rule baseline, you cannot honestly show that ML improved anything.

The rule engine also mirrors the real-world rule-driven AML environment described by the AML literature.

Output

```
rule_score
rule_reasons[]
```

The `rule_reasons` field becomes an excellent UX feature in the dashboard.

Module 08 —XGBoost / Random Forest Baseline

Purpose

Create a strong tabular baseline before introducing graph neural networks.

Inputs

The engineered account/transaction features from Module 05.

Why XGBoost?

- strong tabular baseline;
- fast on moderate data;
- handles non-linear interactions;
- easy to integrate with SHAP;
- widely used in financial ML;
- gives the team a useful benchmark against a GNN.

Output

```
```

```
risk_probability
predicted_label
```

Required experiments

Compare:

```
Rules
vs
XGBoost
```

using the same temporal split and evaluation metrics.

Module 09 —SHAP Explanation Engine

Purpose

Explain the baseline model and provide interpretable feature-level evidence.

Example output

```
{
  "risk": 0.91,
  "top_features": [
    ["cycle_count", 0.22],
    ["out_degree", 0.15],
    ["transaction_burst_score", 0.12],
    ["total_out", 0.09]
  ]
}
```

Explanation sentence generator

Turn structured explanation values into deterministic templates:

```
This account received a high-risk score primarily because it
participates in multiple short transaction cycles, has unusually
high outbound connectivity, and shows a recent increase in
transaction activity.
```

The system should generate this from measured features, not invent evidence.

Module 10 —GraphSAGE Model

Purpose

Learn representations using neighboring account information.

Input

```
node features
edge_index
edge features (if supported by chosen architecture)
```

Minimal architecture

```
Input features
      |
      v
SAGEConv
      |
      v
ReLU
      |
      v
Dropout
      |
      v
SAGEConv
      |
      v
Linear classifier
      |
      v
Risk probability
```

Initial model size

Keep it small:

```
hidden_dim = 64 or 128
layers = 2
dropout = 0.2–0.5
```

These are starting configurations, not claimed optimal values.

Training objective

For binary node classification:

```
Binary Cross Entropy
```

For strong class imbalance, test:

- weighted BCE;
- focal loss if necessary.

Do not jump to exotic losses before establishing a simple baseline.

Module 11 —GNNExplainer Engine

Purpose

Explain which graph neighborhood and node features contributed to a particular prediction.

Example result

```
Target account: A1042
Predicted risk: 0.93

Important graph structure:

A1042 → B2911 → C5120
   |                |
   +--------------> D9912

Important node features:
- recent transaction intensity
- outbound amount
- number of counterparties
```

Expected output

- node mask;
- edge mask;
- feature importance;
- subgraph containing high-contribution edges/nodes.

Visualization

The dashboard should highlight:

- target node;
- top contributing neighboring nodes;
- high-contribution edges;
- transaction amount;
- timestamp;
- direction of flow.

Module 12 —Suspicious Path / Pattern Extractor

Purpose

Convert a raw explanation subgraph into a recognizable financial pattern.

Examples

Cycle

```
A → B → C → A
```

Fan-out

```
      B
     /
A → C
     \
      D
```

Fan-in

```
B → A ← C
    ^
    |
    D
```

Why this module matters

A raw GNN explanation is not necessarily a useful investigator explanation.

The pattern extractor acts as the bridge between machine representation and financial reasoning.

Implementation

Use graph algorithms from NetworkX or PyTorch Geometric utilities to detect:

- short cycles;
- fan-in;
- fan-out;
- high-flow chains;
- repeated paths.

The graph-pattern module should run after detection, not become a giant pre-training research project.

Module 13 —Risk Aggregator

Purpose

Combine multiple signals into an operational case score.

Example:

```
Final Risk
```

```
= model score
+ rule score
+ recent-behavior score
+ graph pattern score
```

Do not invent arbitrary weights and pretend they are statistically justified.

Use one of these approaches:

MVP

Keep model scores separate:

```
ML Risk: 0.87
Rule Risk: 0.62
Pattern Risk: HIGH
```

Advanced

Train a simple calibration/risk-combination model on held-out data.

Module 14 —Case Builder

Purpose

Convert one model prediction into a self-contained investigation case.

Example case schema

```
{
  "case_id": "CASE-00123",
  "target_account": "A1042",
  "risk": 0.93,
  "pattern": "cycle",
  "top_features": [...],
  "important_edges": [...],
  "important_nodes": [...],
  "supporting_transactions": [...],
  "generated_explanation": "..."
}
```

Why this is a major engineering module

It decouples ML from the UI.

The dashboard should consume a structured case rather than directly calling the model.

That makes the system:

- testable;

- reusable;
- easier to debug;
- easier to demo;
- easier to change models later.

Module 15 — Investigator Dashboard

Technology

Streamlit is sufficient for the MVP.

Main screens

Screen A — Alert Queue

```
Case ID     Account    Risk    Pattern      Priority
-----------------------------------------------------
00123       A1042      0.93    Cycle        HIGH
00141       A2871      0.88    Fan-out      HIGH
00146       A8103      0.71    Burst        MEDIUM
```

Screen B — Case Investigation

Show:

- risk score;
- graph visualization;
- top contributing features;
- suspicious transaction timeline;
- suspicious path/cycle;
- model explanation;
- rule explanation.

Screen C — Compare Explanations

For one case:

```
SHAP:
transaction burst +0.18
cycle count +0.15
out degree +0.11

GNNExplainer:
A → B → C → A
```

Screen D — Decision Feedback

```
[ Confirm for investigation ]
[ False positive ]
[ Needs more review ]
```

Store this feedback even if it is only used for later analysis.

---

10. Data Model

10.1 Transactions table

Suggested normalized schema:

Field	Type	Meaning
transaction_id	string	Unique transaction identifier
timestamp	datetime	Transaction time
sender_id	string	Source account
receiver_id	string	Destination account
amount	float	Transaction amount
transaction_type	categorical	WIRE/CREDIT/etc. depending on dataset
bank_id	categorical	Bank identifier where available
label	binary/categorical	Training/evaluation label if available

10.2 Account table

Field	Meaning
account_id	unique account
bank_id	associated bank
account_type	type if available
risk_label	target if generated/known

10.3 Pattern membership table

Field	Meaning
pattern_id	AMLSim pattern identifier
pattern_type	cycle/fan-in/fan-out/etc.
account_id	account involved
transaction_id	transaction involved
is_sar	known synthetic alert/SAR flag when available

11. Model Tasks

The project can support three possible prediction tasks.

11.1 Node classification

> Is this account involved in suspicious activity?

This is the recommended MVP task.

11.2 Edge classification

> Is this transaction suspicious?

Useful if the generated labels are naturally transaction-level.

11.3 Case-level classification

> Is this connected subgraph / alert worth escalating?

Use this only after the first two are stable.

Recommended progression

```
Node classification
        |
        v
Edge scoring
        |
        v
Alert/case prioritization
```

---

12. Evaluation Plan

12.1 Do not use accuracy as the primary metric

AML problems are usually highly imbalanced. A classifier that predicts almost everything as normal can achieve deceptively high accuracy.

Use:

- Precision;
- Recall;
- F1;
- PR-AUC;
- ROC-AUC as a secondary metric;
- false-positive rate;

- precision@K;
- recall@K;
- alert workload reduction.

12.2 Alert prioritization metrics

If the product produces an ordered queue, measure:

Precision@K

```
number of true suspicious cases in top K
----------------------------------------
K
```

Recall@K

```
number of suspicious cases retrieved in top K
---------------------------------------------
total suspicious cases
```

This is more operationally meaningful than only reporting overall F1.

12.3 Explanation metrics

A. Feature overlap

Compare top-k features from repeated explanations.

B. Rank correlation

Use Spearman correlation between explanation rankings.

C. Fidelity / faithfulness

Perturb or remove important features/edges and measure how much the prediction changes.

D. Explanation sparsity

How large is the explanation subgraph?

A useful explanation should not be:

```
entire 30,000-node network
```

E. Ground-truth pattern overlap

This is where AMLSim becomes especially valuable.

If an AMLSim alert was generated from:

```
A → B → C → A
```

measure whether the explanation highlights those nodes/edges.

12.4 System metrics

Measure:

- training time;
- inference latency;
- explanation runtime;
- memory consumption;
- dashboard response time.

---

13. Experimental Design

The project should not consist of one model and one test set.

Experiment 1 —Rule baseline

```
AMLSim
  |
  v
Hand-engineered rules
  |
  v
Alert queue
```

Measure false positives and recall.

Experiment 2 —XGBoost

```
Graph-derived features
        |
        v
     XGBoost
        |
        v
    SHAP
```

Compare with the rule baseline.

Experiment 3 —GraphSAGE

```
Transaction graph
        |
        v
    GraphSAGE
        |
        v
```

```
Account risk
```

Compare with XGBoost.

Experiment 4 —Explain the GNN

Generate GNNExplainer explanations for selected cases.

Experiment 5 —Explanation correctness

Compare GNNExplainer subgraphs to the known AMLSim pattern used to generate the alert.

Experiment 6 —Generalization

Change:

- transaction volume;
- pattern frequency;
- time-window configuration;
- class ratio.

Test whether the model still behaves sensibly.

Experiment 7 —Secondary dataset

If the MVP works:

```
AMLSim-trained pipeline
        |
        v
Elliptic benchmark
```

This should be treated as a separate experiment because the feature semantics and labels differ.

---

14. Recommended Train / Validation / Test Strategy

Do not random-split temporal financial data blindly

If transactions are time dependent, random splitting can leak future behavior into training.

Preferred split:

```
Time ----->

TRAIN        VALIDATE        TEST
|------------|---------------|
older data   middle data     newer data
```

For example:

```
60% earliest timesteps → training
20% next timesteps     → validation
20% latest timesteps   → test
```

The exact split should be decided after inspecting the generated timeline.

Why this matters

A model should ideally learn from information that would have been available at the time of the alert.

This is particularly important for the final report because temporal leakage is an easy weakness for an evaluator to identify.

---

15. Class Imbalance Strategy

AML labels can be extremely skewed.

Use:

1. class-weighted losses;
2. precision-recall curves;
3. threshold tuning on validation data;
4. stratified reporting where appropriate;
5. possibly focal loss for the GNN.

Avoid blindly oversampling graph nodes because graph structure can make naive duplication misleading.

For a first implementation:

```
weighted BCE
+
threshold tuning
+
PR-AUC
```

is enough.

---

16. Explainability Design

GraphGuard should have two layers of explanation.

Layer 1 —Numerical explanation

SHAP answers:

> What engineered features influenced the score?

Example:

```
+0.21 cycle_count
+0.14 outbound_burst
+0.12 unique_counterparties
+0.08 total_out
-0.04 account_age
```

Layer 2 —Structural explanation

GNNExplainer answers:

> What part of the transaction network influenced the GNN's prediction?

Example:

```
A → B → C
|         |
+---------+
```

Then a third deterministic layer translates those findings:

> “The model's high-risk prediction is associated with repeated short-path fund transfers forming a transaction cycle, together with unusually high outbound activity.”

The wording should be generated only from evidence produced by the model and graph-analysis module.

---

17. Explanation Validation

This is one of the most important sections of the project.

An explanation is not automatically trustworthy just because it came from SHAP or GNNExplainer.

The uploaded SHAP/LIME perspective paper specifically warns that SHAP/LIME explanations can be affected by the underlying model and feature collinearity. Therefore, GraphGuard should treat explanations as outputs that need validation, not unquestionable truth.

Source: uploaded `2305.02012v3.pdf` —*A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME*.

Proposed validation checks

Check 1 —Prediction consistency

Repeat explanations for equivalent or minimally perturbed inputs.

Check 2 —Feature stability

Compare top-k SHAP feature rankings.

Check 3 —Structural stability

Compare important edges/subgraphs from GNNExplainer.

Check 4 —Ground truth

On AMLSim, compare the highlighted explanation structure with the known generated laundering pattern.

Check 5 —Faithfulness

Mask important features/edges and observe prediction change.

This gives the project a meaningful XAI evaluation layer without attempting to create a brand-new explanation algorithm.

---

18. Research Questions

Primary research question

> Can graph-based machine learning detect suspicious transaction networks more effectively than tabular baselines while generating explanations that correspond to known suspicious transaction structures?

Secondary questions

RQ2

How much does temporal feature engineering improve AML alert prioritization compared with static graph features?

RQ3

Do GNN explanations identify AMLSim's known transaction structures?

RQ4

How does explanation quality change when graph size and transaction density increase?

RQ5

Can an analyst-facing explanation reduce the amount of transaction data an investigator must inspect manually?

RQ5 should be treated as a potential user-study extension rather than a mandatory scientific claim.

---

19. What Counts as a Successful Project?

A successful MVP does not require state-of-the-art AML detection.

It should demonstrate these five things:

1. Detection: identify suspicious nodes/transactions better than a simple rules baseline on the selected dataset.
2. Graph reasoning: the GNN uses neighborhood structure rather than only individual transaction values.
3. Explanation: the system identifies important features and graph structures.
4. Investigation workflow: an alert can be opened as an explainable case in the dashboard.
5. Validation: explanations are evaluated using measurable criteria and, where possible, known AMLSim ground truth.

---

20. Five-Person Team Allocation

The team should not divide work by “everyone trains a model.” That creates five copies of the same task and nobody integrates anything.

Member 1 —Strongest technical member

Role

Graph ML Lead

Responsibilities

- GraphSAGE architecture;
- GNN training pipeline;
- graph batching/sampling;
- model debugging;
- GNN experiments;
- performance optimization.

Stretch

- GAT;
- heterogeneous graph;
- GraphSAGE inductive evaluation.

Member 2 —You

Role

ML + XAI + Research Lead

Responsibilities

- feature engineering design;
- XGBoost baseline;
- SHAP;
- GNNExplainer integration;
- explanation metrics;
- experiment design;
- ablation studies;
- final research narrative.

This gives you enough depth to rebuild your ML fundamentals through the project rather than becoming the team's human Jira board.

Member 3 —Data Engineer

Responsibilities

- AMLSim setup;
- dataset generation;
- cleaning;
- schema normalization;
- train/validation/test generation;
- reproducibility scripts;
- data documentation.

Success condition

Anyone on the team can run one command and regenerate the project's processed dataset.

Member 4 —Graph / Backend Engineer

Responsibilities

- graph construction;
- NetworkX utilities;
- suspicious path/cycle extraction;
- case-building API/data structures;

- model output serialization.

Stretch

- temporal graph utilities;
- graph caching;
- explanation subgraph extraction.

Member 5 —Frontend / QA / Integration

Responsibilities

- Streamlit dashboard;
- transaction-network visualization;
- alert queue;
- case page;
- explanation visualization;
- end-to-end testing;
- demo scripts;
- documentation.

Stretch

- analyst feedback logging;
- explanation comparison UI;
- exportable investigation reports.

---

21. Repository Structure

Recommended repository:

```
GraphGuard-AML/
│
├── README.md
├── requirements.txt
├── pyproject.toml
│
├── configs/
│   ├── amlsim_small.json
│   ├── amlsim_medium.json
│   └── model.yaml
│
├── data/
│   ├── raw/
│   ├── interim/
```

```
│   ├── processed/
│   └── README.md
│
├── src/
│   ├── ingestion/
│   │   ├── load_transactions.py
│   │   └── validate.py
│   │
│   ├── graph/
│   │   ├── build_graph.py
│   │   ├── temporal_windows.py
│   │   └── graph_features.py
│   │
│   ├── features/
│   │   ├── account_features.py
│   │   └── transaction_features.py
│   │
│   ├── labels/
│   │   └── amlsim_labels.py
│   │
│   ├── baselines/
│   │   ├── rules.py
│   │   └── xgboost_model.py
│   │
│   ├── gnn/
│   │   ├── graphsage.py
│   │   ├── train.py
│   │   └── evaluate.py
│   │
│   ├── explainability/
│   │   ├── shap_explainer.py
│   │   ├── gnn_explainer.py
│   │   └── explanation_metrics.py
│   │
│   ├── investigation/
│   │   ├── pattern_detector.py
│   │   ├── case_builder.py
│   │   └── explanation_text.py
│   │
│   └── utils/
│       ├── logging.py
│       ├── seeds.py
│       └── io.py
│
├── dashboard/
│   ├── app.py
│   ├── pages/
│   └── components/
│
├── experiments/
│   ├── configs/
│   ├── results/
```

```
│   └── reports/
│
├── tests/
│   ├── test_ingestion.py
│   ├── test_graph.py
│   ├── test_features.py
│   └── test_case_builder.py
│
└── docs/
    ├── architecture.md
    ├── dataset.md
    └── experiments.md
```

---

22. API / Module Contracts

Every major module should have a stable input/output contract.

Example

```python
def build_transaction_graph(
    transactions: "pd.DataFrame",
    account_features: "pd.DataFrame | None" = None,
):
    """Return a PyTorch Geometric graph object and ID mappings."""
```

Returned structure

```
{
  "graph": pyg_data,
  "account_to_idx": {...},
  "idx_to_account": {...}
}
```

The UI should never need to understand how the graph was built.

Likewise:

```python
def explain_case(case_id: str) → dict:
    ...
```

should return a structured explanation consumed by the dashboard.

---

23. 12-Week Execution Plan

Week 1 —Problem + environment

- finalize task definition;
- clone AMLSim;
- establish Python environment;
- create project repository;
- generate a tiny dataset;
- document data schema.

Deliverable: everyone can run the pipeline and inspect transactions.

Week 2 —Data pipeline

- ingestion;
- cleaning;
- labels;
- deterministic splits;
- exploratory graph statistics.

Deliverable: clean dataset + graph statistics report.

Week 3 —Graph construction

- NetworkX graph;
- PyG graph;
- node/edge mappings;
- simple network visualizations.

Deliverable: working graph object.

Week 4 —Rule baseline

- rules;
- basic alert queue;
- metrics;
- initial dashboard.

Deliverable: non-ML AML alert system.

Week 5 —XGBoost

- feature engineering;
- model training;
- threshold tuning;
- PR-AUC/F1/recall evaluation.

Deliverable: strong tabular baseline.

Week 6 —SHAP

- global explanation;

- local explanation;
- explanation JSON schema;
- first natural-language templates.

Deliverable: explainable XGBoost alerts.

Week 7 — GraphSAGE

- PyG model;
- training loop;
- imbalance handling;
- baseline comparison.

Deliverable: working GNN detector.

Week 8 — GNNExplainer

- explanation generation;
- edge/node masks;
- explanation visualization.

Deliverable: graph-level investigation explanation.

Week 9 — Pattern extraction

- cycles;
- fan-in/fan-out;
- suspicious paths;
- compare explanation subgraphs with AMLSim labels.

Deliverable: interpretable case structures.

Week 10 — Case dashboard

- alert queue;
- case page;
- explanation page;
- transaction timeline;
- graph visualization.

Deliverable: end-to-end demo.

Week 11 — Evaluation

- ablations;
- threshold analysis;
- explanation stability;
- runtime measurements;
- larger dataset test.

Deliverable: final experiment table.

Week 12 — Final hardening

- reproduce all experiments;
- clean code;
- prepare figures;
- write report;
- record demo;
- prepare viva questions.

Deliverable: final project.

---

24. Month 4: Hardening and Stretch Work

Once the complete MVP works, select one or two stretch modules.

Stretch A — Temporal GNN

Compare:

```
Static GraphSAGE
vs
Temporal model
```

Stretch B — Heterogeneous graph

Represent:

```
Person
Organization
Account
Transaction
Bank
```

with typed relationships.

This is directly motivated by the heterogeneous MPNN AML work.

Stretch C — Semi-supervised learning

Reduce labeled examples and test whether graph structure can compensate for fewer labels.

Stretch D — Elliptic transfer

Evaluate the detection pipeline on the Elliptic Bitcoin dataset.

Stretch E — Analyst study

Have participants inspect alerts with:

1. risk score only;
2. risk score + SHAP;
3. risk score + graph explanation.

Measure:

- decision accuracy;
- response time;
- confidence;
- ability to reject incorrect alerts.

This becomes a proper human-centered XAI extension.

---

25. MVP vs Stretch Matrix

Component	MVP	Stretch
AMLSim	Yes	Larger simulations
Account graph	Yes	Heterogeneous graph
Temporal features	Yes	Temporal GNN
Rule baseline	Yes	Learned rule combination
XGBoost	Yes	Ensemble
SHAP	Yes	SHAP/LIME comparison
GraphSAGE	Yes	GAT
GNNExplainer	Yes	PGExplainer
Pattern detector	Yes	Advanced typology mining
Dashboard	Yes	Analyst study
Elliptic	No	Yes
Self-supervised learning	No	Yes
Cryptocurrency-specific model	No	Yes

---

26. Main Risks and Mitigations

Risk 1 —AMLSim setup becomes the project

Mitigation

Use a small existing AMLSim configuration first. Do not spend two weeks redesigning the simulator.

Risk 2 — GNN is unstable

Mitigation

Keep XGBoost as a guaranteed baseline.

If GNN training becomes problematic, the project still has:

```
Rules + XGBoost + SHAP + graph investigation
```

which is already a functional system.

Risk 3 — Class imbalance produces misleading performance

Mitigation

Use PR-AUC, Precision@K, Recall@K, F1, threshold tuning and class-weighted loss.

Risk 4 — Synthetic data is too artificial

Mitigation

Say so explicitly in the report.

Then use the Elliptic dataset as a secondary benchmark if time allows.

Do not pretend synthetic banking data is equivalent to a live bank's fraud stream.

Risk 5 — Explanation looks nice but is meaningless

Mitigation

Validate explanations against:

- known AMLSim pattern members;
- perturbation-based faithfulness;
- explanation stability;
- subgraph sparsity.

Risk 6 — Data leakage

Mitigation

Use chronological train/validation/test splits and ensure engineered features use only information available before the prediction time.

Risk 7 —Dashboard consumes the final month

Mitigation

Build a deliberately ugly dashboard by Week 4.

Polish it later.

---

27. Suggested Technology Stack

Python

Core language.

pandas / NumPy

Data processing.

NetworkX

Graph analysis and suspicious-pattern extraction.

PyTorch

GNN training.

PyTorch Geometric

Graph data structures, GNN layers and graph operations.

scikit-learn

Baselines, metrics, preprocessing.

XGBoost

Tabular baseline.

SHAP

Feature-level explanations.

Streamlit

Investigator dashboard.

Plotly

Interactive charts and timelines.

Git

Version control.

pytest

Testing.

Optional

MLflow or simple JSON/CSV experiment logging.

Do not add ten MLOps products to a final-year project that has five humans and twelve weeks.

---

28. Suggested Minimal Dependencies

```
python
pandas
numpy
scikit-learn
xgboost
networkx
matplotlib
plotly
streamlit
shap
torch
torch-geometric
pytest
pyarrow
```

AMLSim itself has separate Java/Python dependencies and should be treated as a data-generation component rather than mixed into the model environment unnecessarily.

---

29. Example End-to-End Run

The demo should look roughly like this:

Step 1 — Generate data

```
python scripts/generate_amlsim.py --config configs/amlsim_small.json
```

Step 2 — Process

```
python -m src.ingestion.load_transactions
python -m src.graph.build_graph
python -m src.features.account_features
```

Step 3 — Train baseline

```
python -m src.baselines.xgboost_model
```

Step 4 — Train GNN

```
python -m src.gnn.train
```

Step 5 — Explain

```
python -m src.explainability.shap_explainer
python -m src.explainability.gnn_explainer
```

Step 6 — Build cases

```
python -m src.investigation.case_builder
```

Step 7 — Launch dashboard

```
streamlit run dashboard/app.py
```

---

30. Example Investigator Workflow

An analyst opens GraphGuard.

Alert Queue

```
A1042     0.93     HIGH
A8771     0.89     HIGH
A4421     0.72     MEDIUM
```

They click A1042.

Case summary

```
Account: A1042
Risk: 0.93

Detected pattern:
Cycle-like transaction flow
```

```
Recent activity:
17 transactions in 24h
11 outbound counterparties
3 repeated short cycles
```

SHAP

```
cycle_count            +0.21
out_degree              +0.15
recent_burst            +0.12
total_out               +0.09
account_age             -0.03
```

Graph explanation

```
A1042 → B2911 → C5120
   ^                |
   |________________|
```

Timeline

```
10:02  A1042 → B2911    ₹90,000
10:41  B2911 → C5120    ₹88,000
11:15  C5120 → A1042    ₹85,000
```

Human-readable evidence summary

```
The account received a high-risk score primarily because it
participates in repeated short transaction cycles and shows
unusually concentrated outbound activity. The explanation
subgraph highlights three connected accounts that form the
main suspicious flow.
```

That is the final product concept.

---

31. Report / Research Paper Structure

Chapter 1 —Introduction

- AML problem;
- limitations of rule-based alerting;
- graph-based detection;
- explainability problem;
- project objectives.

Chapter 2 —Literature Review

Organize it as:

1. Rule-based AML;
2. classical ML AML;
3. graph-based AML;
4. temporal graph AML;
5. explainable AML;
6. GNN explainability;
7. research gap.

Chapter 3 —Problem Definition

Formalize:

```
Given transaction graph G = (V, E, X)
identify suspicious nodes/edges
and produce explanation E_xai
```

Chapter 4 —Dataset and Preprocessing

- AMLSim;
- optional Elliptic;
- graph construction;
- temporal splits;
- class distribution.

Chapter 5 —Baseline Models

- rule engine;
- logistic regression if desired;
- XGBoost.

Chapter 6 —Graph Model

- GraphSAGE;
- node representation;
- message passing;
- training;
- class imbalance.

Chapter 7 —Explainability

- SHAP;
- GNNExplainer;
- explanation generation;

- pattern extraction;
- validation.

Chapter 8 — System Design

- case builder;
- dashboard;
- end-to-end workflow.

Chapter 9 — Experiments and Results

- model comparison;
- temporal experiments;
- explanation metrics;
- ground-truth pattern overlap;
- performance.

Chapter 10 — Discussion

- what worked;
- what failed;
- limits of synthetic data;
- explanation limitations;
- generalization.

Chapter 11 — Conclusion and Future Work

Potential future work:

- heterogeneous graphs;
- temporal GNNs;
- self-supervised AML;
- analyst studies;
- real-world bank integrations.

---

32. Expected Research Contribution

The project should make a modest but defensible contribution.

Do not claim:

> “We invented a new AML algorithm.”

Instead claim something of the form:

> “We implemented and evaluated an explainable graph-based AML investigation pipeline that combines graph learning, feature-level explanations, subgraph explanations and known

> synthetic AML typologies, with an analyst-facing case representation."

A possible empirical contribution is:

> "We evaluate whether explanation subgraphs recover known AMLSim transaction patterns and whether temporal graph features improve alert prioritization relative to static/tabular baselines."

That is a strong final-year project contribution without pretending the five of you solved global money laundering.

---

33. What the Literature Suggests the Project Should Focus On

Across the papers reviewed, several themes repeat:

1. False positives matter

AML alerting is not just a classification problem. Operational workload matters.

2. Relationships matter

A transaction graph can contain information that individual transaction rows cannot.

3. Time matters

Financial activity evolves, and transaction windows can reveal patterns that static aggregates hide.

4. Labels are scarce

This motivates semi-supervised and self-supervised approaches in the broader literature.

5. Explanations matter

An AML analyst needs reasons and evidence, not simply a probability.

6. Explanation quality itself must be evaluated

The SHAP/LIME literature warns against treating explanations as automatically faithful, especially in the presence of model dependence and feature correlations.

These themes should become the five pillars of the final project.

---

34. Recommended Final MVP Definition

Freeze the MVP as:

```
AMLSim
  ↓
Account transaction graph
  ↓
Temporal + graph features
  ↓
Rules + XGBoost
  ↓
GraphSAGE
  ↓
SHAP + GNNExplainer
  ↓
Pattern extractor
  ↓
Case builder
  ↓
Streamlit investigation dashboard
```

Minimum research output

1. Rules vs XGBoost vs GraphSAGE.
2. Static vs temporal features.
3. SHAP feature stability.
4. GNNExplainer subgraph quality.
5. Explanation vs AMLSim ground truth.
6. Precision@K / Recall@K / PR-AUC.

Minimum product output

An investigator can:

1. see prioritized alerts;
2. open one alert;
3. inspect the transaction graph;
4. see why the model flagged it;
5. inspect the suspicious flow;
6. export the case summary.

---

35. Paper / Dataset / Repository Reference List

Primary AML papers

1. Weber et al., Scalable Graph Learning for Anti-Money Laundering: A First Look
   https://arxiv.org/abs/1812.00076
2. Eddin et al., Anti-Money Laundering Alert Optimization Using Machine Learning with Graphs
   https://arxiv.org/abs/2112.07508

3. Cardoso et al., LaundroGraph: Self-Supervised Graph Representation Learning for Anti-Money Laundering
https://arxiv.org/abs/2210.14360
4. Assumpção et al., DELATOR: Money Laundering Detection via Multi-Task Learning on Large Transaction Graphs
https://arxiv.org/abs/2205.10293
5. Alarab & Prakoonwit, Graph-Based LSTM for Anti-money Laundering: Experimenting Temporal Graph Convolutional Network with Bitcoin Data
https://link.springer.com/article/10.1007/s11063-022-10904-8
6. Karim et al., Catch Me If You Can: Semi-supervised Graph Learning for Spotting Money Laundering
https://arxiv.org/abs/2302.11880
7. Johannessen & Jullum, Finding Money Launderers Using Heterogeneous Graph Neural Networks
https://arxiv.org/abs/2307.13499
8. Poon et al., LineMVGNN: Anti-Money Laundering with Line-Graph-Assisted Multi-View Graph Neural Networks
https://arxiv.org/abs/2603.23584
9. Zola et al., Synthetic Pattern Generation and Detection of Financial Activities using Graph Autoencoders
https://www.sciencedirect.com/science/article/pii/S1877050926011798
10. Lu & Wang, Graph Contrastive Pre-training for Anti-money Laundering
https://link.springer.com/article/10.1007/s44196-024-00720-4
11. HeteroGCL: A Heterogeneous Graph Contrastive Learning Framework for Scalable and Sustainable Cryptocurrency AML
https://www.mdpi.com/2076-3417/16/6/2860
12. Shang, Digital currency money laundering identification model based on graph structure and transformer self-supervised learning
https://link.springer.com/article/10.1007/s44163-026-02264-2

Foundational graph papers

13. Hamilton, Ying & Leskovec, Inductive Representation Learning on Large Graphs (GraphSAGE)
https://arxiv.org/abs/1706.02216
14. Ying et al., GNNExplainer: Generating Explanations for Graph Neural Networks
https://arxiv.org/abs/1903.03894
15. Kipf & Welling, Semi-Supervised Classification with Graph Convolutional Networks
https://arxiv.org/abs/1609.02907
16. Pareja et al., EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs
https://arxiv.org/abs/1902.10191

Foundational XAI papers

17. Lundberg & Lee, A Unified Approach to Interpreting Model Predictions
https://arxiv.org/abs/1705.07874
18. Ribeiro, Singh & Guestrin, “Why Should I Trust You?”: Explaining the Predictions of Any Classifier

https://arxiv.org/abs/1602.04938

19. Molnar, Interpretable Machine Learning
https://christophm.github.io/interpretable-ml-book/

Relevant repositories

20. IBM AMLSim
https://github.com/IBM/AMLSim
21. Original GNNExplainer implementation
https://github.com/RexYing/gnn-model-explainer
22. Heterogeneous MPNN implementation used for AML work
https://github.com/fredjo89/heterogeneous-mpnn
23. PyTorch Geometric
https://github.com/pyg-team/pytorch_geometric
24. Graph/fraud research bibliography
https://github.com/safe-graph/graph-fraud-detection-papers
25. GNN-for-AML paper list
https://github.com/fuqishuang228/GNN-for-Anti-money-launedring

Datasets

26. IBM AMLSim synthetic AML generator/data source
https://github.com/IBM/AMLSim
27. Elliptic Bitcoin transaction graph dataset
https://www.kaggle.com/ellipticco/elliptic-data-set

---

36. Sources Supplied by the Project Team

The project was designed using the uploaded research material as part of the literature base:

- `Introduction to xAI.md`
- `xAI in Cyber Security.md`
- `xAI in Finance.md`
- `2305.02012v3.pdf` —*A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME*
- `Research-able_Projects?[CyberSecurity].md`
- `Research-able_Projects?[Finance].md`
- `frai-07-1236947.pdf` —*The privacy-explainability trade-off: unraveling the impacts of differential privacy and federated learning on attribution methods*

For GraphGuard, the most directly relevant supplied documents are the finance XAI review and the finance project research notes because they explicitly identify AML as an area where XAI and graph-based financial analysis can be useful.

---

37. Final Recommendation for Implementation Order

Do not build everything in parallel.

The dependency chain should be:

```
1. AMLSim data
       ↓
2. normalized transactions
       ↓
3. graph
       ↓
4. graph features
       ↓
5. rule baseline
       ↓
6. XGBoost
       ↓
7. SHAP
       ↓
8. GraphSAGE
       ↓
9. GNNExplainer
       ↓
10. pattern extraction
       ↓
11. case builder
       ↓
12. dashboard
```

The non-negotiable checkpoint is Week 5:

```
Dataset works
Graph works
Rules work
XGBoost works
Evaluation works
```

If those five pieces are not functioning by then, do not add GAT, Transformers, self-supervised learning, heterogeneous graphs or anything else with a suspiciously impressive acronym.

---

38. One-Sentence Project Definition for Faculty

> GraphGuard AML is an explainable graph-learning system that models financial transactions as a dynamic network, prioritizes potentially suspicious accounts or transactions, and provides evidence-based structural and feature-level explanations to assist AML investigators in reviewing alerts.

39. One-Sentence Research Gap

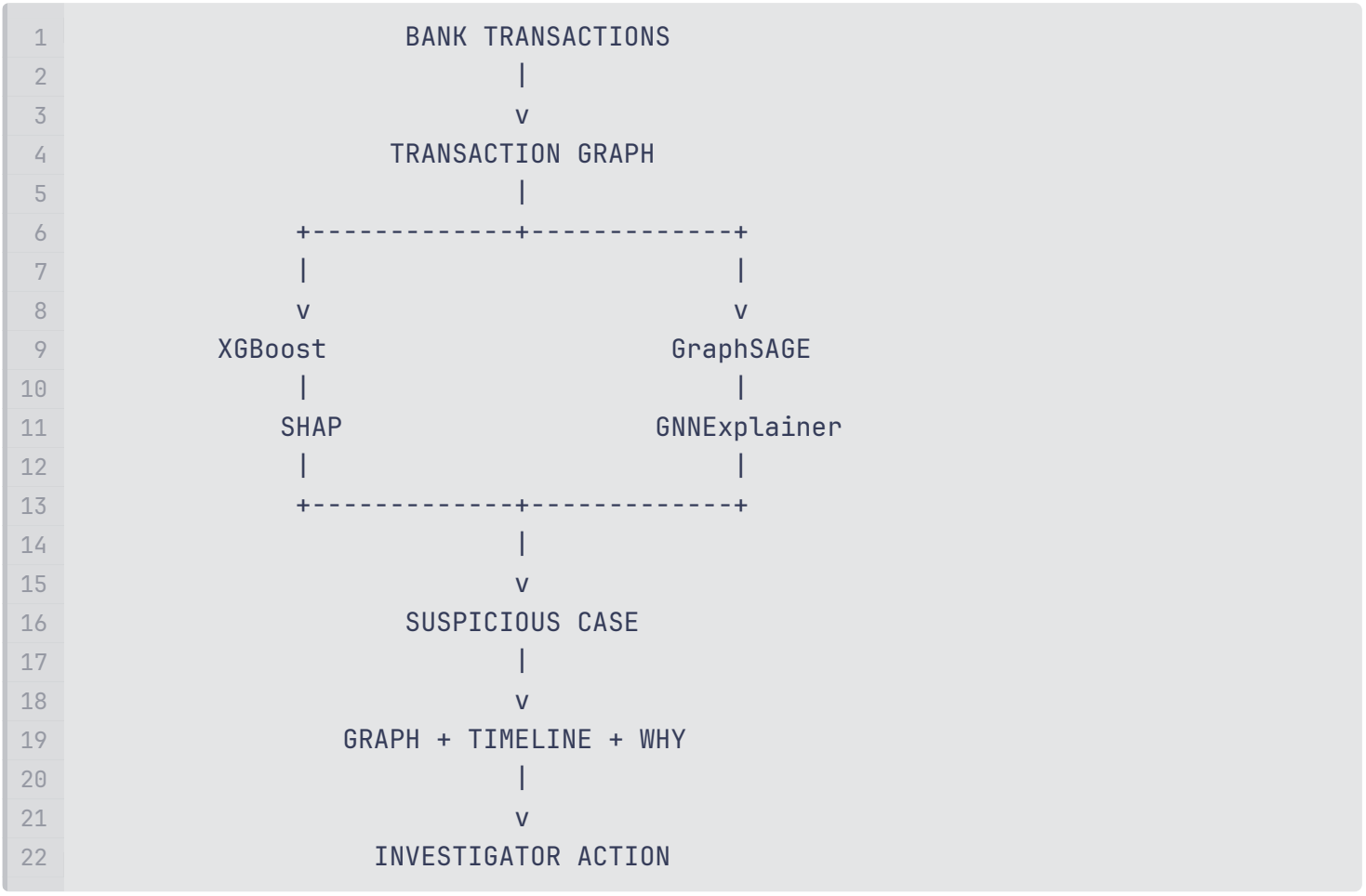
> Existing AML research increasingly uses graph learning for detection, but GraphGuard focuses on connecting graph-based risk detection with validated, investigator-oriented explanations and known synthetic laundering patterns in an end-to-end student-implementable workflow.

This statement should be checked against the final literature review before submission. It is deliberately narrower than claiming that no prior work has attempted explainable graph-based AML.

---

40. Final Build Target

At the end of the project, the examiner should be able to watch this happen:



The project is complete when the system can make that entire journey reproducibly, and when the team can defend every arrow in the diagram with an experiment.